

Company-wise Question Bank

TCS, Deloitte, Goldman Sachs & more · CrackAnalytics · by Mahendra Singh

Explore more — click on this link: <https://medium.com/@mahendraa1188>

IPG Mediabrands

Q1. Explain your project? [Easy]

Use the 6-step framework from the Projects tab: business problem → data sources → tools → your process (cleaning, modeling, DAX) → 2–3 insights with numbers → business impact. Keep it to 2 minutes.

Q2. Power BI pipeline — explain [Medium]

End-to-end flow: **Data sources** → **Power Query (ETL: clean/transform)** → **Data model (star schema, relationships)** → **DAX measures** → **Report visuals** → **Publish to Power BI Service** → **Scheduled refresh via Gateway** → **Share via workspace/app to stakeholders**.

Q3. What is your file size in Power BI? [Easy]

Give a realistic number: "Around 150–400 MB .pbix for ~1–2 million rows after removing unused columns." Add that you keep it small by dropping high-cardinality columns, aggregating where possible, and using incremental refresh for big fact tables.

Q4. What visualizations did you use in Power BI? [Easy]

Cards/KPIs for headline numbers, bar/column for category comparison, line for trends, matrix for detailed cross-tabs, map for regional view, slicers for filtering, plus drill-through pages and tooltips for detail. Justify each with its purpose.

Q5. How to connect a database with Power BI? [Easy]

Home → Get Data → choose the connector (SQL Server etc.) → enter server + database → pick Import or DirectQuery → authenticate (Windows/DB credentials) → select tables or paste a SQL query → Load/Transform. For refresh in the Service, configure an **On-premises Data Gateway**.

Q6. What is DAX? Types of DAX? [Medium]

DAX (Data Analysis Expressions) is Power BI's formula language. Three usage types: **Calculated columns** (row-by-row, stored), **Measures** (aggregations at query time), **Calculated tables**. Function families: aggregation (SUM, AVERAGE), filter (CALCULATE, FILTER, ALL), time intelligence (TOTALYTD, SAMEPERIODLASTYEAR), logical, text, and relationship functions (RELATED).

Q7. What DAX measures have you created? [Medium]

Example set: Total Sales = SUM(Sales[Amount]); Profit Margin % = DIVIDE([Profit], [Total Sales]); Sales LY = CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date])); YoY % = DIVIDE([Total Sales] - [Sales LY], [Sales LY]); running totals with TOTALYTD. Name only measures you can write on a whiteboard.

Q8. Difference between VALUES and SELECTEDVALUE [Medium]

VALUES(column) returns a **table** of the distinct values visible in the current filter context (can be many rows). SELECTEDVALUE(column, alt) returns a **single value** if exactly one value is selected, otherwise the alternate (or blank) — perfect for showing a slicer's selection in a title.

Q9. Difference between FILTERS and FILTER? Which is faster? [Hard]

FILTER(table, condition) is an **iterator** that scans a table row by row and returns a filtered table — powerful but slower. FILTERS(column) just returns the values currently being directly filtered on a column. For performance, a simple boolean filter inside CALCULATE (e.g. Region="A") is faster than wrapping FILTER around a whole table, because it works on the column, not row-by-row.

Q10. PARALLELPERIOD vs SAMEPERIODLASTYEAR — difference and required dimensions [Hard]

Both need a proper **Date dimension** marked as a date table. SAMEPERIODLASTYEAR shifts the current selection exactly one year back, day-for-day. PARALLELPERIOD(dates, -1, YEAR) returns the **full parallel period** (whole previous year/quarter/month), ignoring partial selections — so for "same dates last year" use SPLY; for "entire previous period" use

PARALLELPERIOD.

Q11. Which is faster: SUM with FILTER(region='A') vs a simple region filter — and how to handle empty age data? [Hard]

The **simple boolean filter** — `CALCULATE(SUM(Sales[Amt]), Sales[Region]="A")` — is faster; it's translated into an efficient column filter, while `FILTER(Sales, ...)` iterates the whole table row by row. For empty age data: handle at source/Power Query (replace nulls with a default or "Unknown" bucket), or in DAX with `COALESCE([Age], 0) / IF(ISBLANK(...))` depending on whether blank should mean zero or excluded.

Q12. SQL Q1. What are window functions and their use cases? [Medium]

Functions that compute over a "window" of related rows without collapsing them: `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LEAD`, `LAG`, `SUM()` `OVER()`. Use cases: rankings/top-N per group, running totals, comparing a row with the previous row (month-over-month), de-duplication with `ROW_NUMBER`.

Q13. SQL Q2. Top 5 customers according to marks — how to calculate? [Medium]

```
SELECT * FROM (
    SELECT customer, marks,
           DENSE_RANK() OVER (ORDER BY marks DESC) AS rnk
    FROM student
) t
WHERE rnk <= 5;
```

`DENSE_RANK` handles ties gracefully; use `ROW_NUMBER` if exactly 5 rows are needed regardless of ties.

Q14. SQL Q3. What will COUNT(*) from table abc return? [Easy]

The **total number of rows** in the table — including duplicates and rows containing NULLs (`COUNT(*)` never ignores NULL rows, unlike `COUNT(column)`).

Q15. SQL Q4. Difference between COUNT(*) and COUNT(1)? [Easy]

No practical difference — both count all rows including NULLs, and modern optimizers produce identical plans. The real distinction is with `COUNT(column)`, which skips NULLs in that column.

Congesool

Q1. Explain your project [Easy]

Same 6-step structure: problem → data → tools → process → insights → impact. Tailor the domain if you know the company's clients.

Q2. What DAX measures did you create in your project? [Medium]

Total Sales, Profit %, YoY growth with `SAMEPERIODLASTYEAR`, rolling 3-month average with `DATESINPERIOD`, and Top-N ranking with `RANKX` — pick 4–5 you can write live.

Q3. Difference between VALUES and SELECTEDVALUE [Medium]

`VALUES` returns a table of distinct visible values; `SELECTEDVALUE` returns the single selected value or an alternate if multiple/none — ideal for dynamic titles like "Sales for " & `SELECTEDVALUE(Region[Name], "All Regions")`.

Q4. How did you optimize your Power BI dashboards? [Hard]

Star schema, removed unused columns, measures instead of calculated columns, single-direction relationships, limited visuals per page, page-level filters, incremental refresh on the fact table, and verified with Performance Analyzer — quote before/after load time if you can.

Q5. Explain how you implemented RLS in your project [Hard]

Created roles in Modeling → Manage Roles with a DAX rule, e.g. `[Region] = USERPRINCIPALNAME()` mapped via a user-region table; tested with "View As Role"; assigned members to roles in the Service. Mention **dynamic RLS** (one rule, user table) vs static roles.

Q6. Explain the TREATAS function [Hard]

TREATAS(values, column) applies a set of values as a filter on a column **as if a relationship existed** — the standard way to filter across tables with no physical relationship. Example: CALCULATE([Sales], TREATAS(VALUES(Budget[Region]), Sales[Region])).

Q7. SUM(sales, FILTER(region='A')) vs SUM with direct region filter — which is faster? [Hard]

The direct boolean filter inside CALCULATE is faster — it filters the column via the storage engine, while FILTER() iterates row by row in the formula engine. Use FILTER only when the condition genuinely needs row context (comparing columns to each other).

TransUnion

Q1. What are the main 3 KPIs in your project? [Easy]

Have three ready with definitions, e.g.: **Total Revenue** (SUM of sales), **Profit Margin %** (profit/revenue), **YoY Growth %** — and one sentence on why each mattered to the business. Domain-fit KPIs (churn %, approval rate) are even better.

Q2. Explain DAX which you created in your project [Medium]

Walk through one measure end-to-end, e.g. YoY%: base measure → CALCULATE with SAMEPERIODLASTYEAR → DIVIDE for safe division → why DIVIDE over "/" (handles divide-by-zero). Depth on one beats listing ten.

Q3. Explain where you used RLS in the project [Hard]

Example: "Regional managers should see only their region — I built dynamic RLS with a UserAccess table (email ↔ region), rule [Region] IN CALCULATETABLE(VALUES(UserAccess[Region]), UserAccess[Email] = USERPRINCIPALNAME()), tested via View As Role."

Q4. SQL Q1. Count the number of commas in the string 'a,b,c,d,e' [Medium]

```
SELECT LENGTH('a,b,c,d,e')
      - LENGTH(REPLACE('a,b,c,d,e', ',', '')) AS comma_count;
-- SQL Server: use LEN() instead of LENGTH() → returns 4
```

Classic trick: total length minus length without commas = number of commas.

Q5. SQL Q2. Top 3 customers by month and revenue using a window function [Medium]

```
SELECT * FROM (
    SELECT month, customer, revenue,
           DENSE_RANK() OVER (PARTITION BY month
                               ORDER BY revenue DESC) AS rnk
    FROM sales
) t
WHERE rnk <= 3;
```

PARTITION BY month restarts the ranking every month — top 3 per month.

Q6. SQL Q3. Which window function did you use in your project and why? [Medium]

Example answer: "ROW_NUMBER to de-duplicate records keeping the latest per customer, and LAG to compute month-over-month change — both avoided messy self-joins and kept queries readable."

Scatter Pie (Tableau practical)

Q1. Explain your project and role as Tableau developer [Easy]

Structure: business problem → sources → how you modeled/joined data → dashboards built (name the charts) → interactivity (filters, actions, parameters) → performance work → impact. Emphasize the *developer* parts: calculations, LODs, publishing, refresh schedules.

Q2. Show me a pie chart (practical) [Easy]

Drag the dimension to Color and the measure to Angle on the Marks card (set mark type to Pie) — or select dimension + measure and hit the pie in Show Me. Add labels (dimension + measure, quick table calc → percent of total).

Q3. Drag customer name to Rows and show the last name in another column (practical) [Medium]

Create a calculated field: Last Name = TRIM(SPLIT([Customer Name], " ", -1)) — SPLIT with index -1 takes the last token. Drag it next to Customer Name on Rows.

Q4. Show me last month's last day (practical) [Medium]

Calculated field: DATEADD('day', -1, DATETRUNC('month', TODAY())) — truncate today to the 1st of this month, minus one day = last day of previous month. Use it in a filter or as a reference.

Q5. Explain how you did optimization in your Tableau project [Hard]

Moved live connections to extracts (aggregated, filtered), reduced quick filters and replaced "Only relevant values" where costly, cut marks per view, replaced heavy table calcs with LODs or source-side logic, and verified with Performance Recording — mention a before/after if possible.

Q6. Explain RLS — how did you implement it in the project? [Hard]

Entitlement-table approach: joined a UserAccess table, data source filter USERNAME() = [User Email]; published with the filter locked so every viewer sees only their rows. Contrast with manual user filters and say why the calculated approach scales.

Q7. Power BI Q1. Limitations of DirectQuery? [Medium]

Slower visuals (every interaction = live query), limited Power Query transformations, several DAX functions restricted or slow (time intelligence needs a date table), 1-million-row limit per query result, no calculated tables on DQ sources, and total dependence on source database performance.

Q8. Power BI Q2. Write a running total calculation [Medium]

```
Running Total =  
CALCULATE(  
    SUM(Sales[Amount]),  
    FILTER(  
        ALLSELECTED('Date'[Date]),  
        'Date'[Date] <= MAX('Date'[Date])  
    )  
)
```

Or simply TOTALYTD(SUM(Sales[Amount]), 'Date'[Date]) for a year-bounded running total.

Q9. Power BI Q3. Write a DAX for Dealer Revenue [Easy]

```
Dealer Revenue =  
CALCULATE(  
    SUM(Sales[Amount]),  
    Sales[Channel] = "Dealer"  
)
```

Pattern: base aggregation + a filter on the channel/segment column inside CALCULATE.

Q10. Power BI Q4. Cumulative sum DAX [Medium]

```
Cumulative Sales =  
CALCULATE(  
    SUM(Sales[Amount]),  
    FILTER(ALL('Date'[Date]), 'Date'[Date] <= MAX('Date'[Date]))  
)
```

ALL removes the date filter, then we re-filter to "all dates up to the current one" — the classic cumulative pattern.

Q11. Power BI Q5. DAX for percentage of total [Medium]

```
% of Total =  
DIVIDE(  
    SUM(Sales[Amount]),  
    CALCULATE(SUM(Sales[Amount]), ALL(Sales))  
)
```

Numerator respects current filters; denominator ignores them via ALL — format as percentage. Use ALLSELECTED instead of ALL to respect slicer selections.

ZS Associates

Q1. Table x has (1,1,1,1,1) and table z has (1,1,1). Output of LEFT / RIGHT / INNER / FULL join? [Hard]

Every row of x matches every row of z (all values are 1), so joins produce a **cartesian match**:

- **INNER JOIN**: $5 \times 3 = 15$ rows
- **LEFT JOIN**: every x row has matches → **15 rows**
- **RIGHT JOIN**: every z row has matches → **15 rows**
- **FULL JOIN**: no unmatched rows on either side → **15 rows**

This is one of the most famous trick questions — remember: duplicate join keys multiply.

Q2. Difference between UNION and UNION ALL? [Easy]

UNION combines results and **removes duplicates** (implicit sort/dedup cost); UNION ALL keeps everything, so it's **faster** — use UNION ALL whenever duplicates are impossible or acceptable.

Q3. Difference between TRUNCATE and DELETE? [Easy]

DELETE removes rows one by one, supports WHERE, is fully logged and can be rolled back; TRUNCATE instantly removes **all** rows, keeps the structure, resets identity, minimal logging, no WHERE. TRUNCATE is DDL-like and much faster for emptying a table.

Goldman Sachs

Q1. What does filter context in DAX mean? [Medium]

Filter context is the **set of filters active on a calculation at evaluation time** — coming from slicers, rows/columns of visuals, page filters and CALCULATE. The same measure returns different values in each cell because each cell has a different filter context. CALCULATE is the function that *modifies* filter context.

Q2. How to implement Row-Level Security (RLS) in Power BI? [Hard]

Modeling → Manage Roles → create a role with a DAX rule (e.g. [Region] = "west" for static, or [Email] = USERPRINCIPALNAME() against a user table for dynamic RLS) → test with View As Role → publish → in the Service, assign users/groups to the role under dataset Security.

Q3. Describe different types of filters in Power BI [Easy]

Visual-level (one visual), **page-level** (all visuals on a page), **report-level** (whole report), **drill-through filters** (carried to a detail page), **slicers** (user-facing), and **cross-filtering** between visuals. Plus filters inside DAX via CALCULATE.

Q4. Difference between ALL and ALLSELECTED in DAX? [Hard]

ALL(table/column) removes **every filter** — grand total regardless of slicers. ALLSELECTED() removes only the filters inside the visual but **respects outside selections** (slicers/page filters). % of total with ALL = share of everything; with ALLSELECTED = share of what the user selected.

Q5. Total sales for a specific product using DAX? [Easy]

```
Laptop Sales =  
CALCULATE(  
    SUM(Sales[Amount]),  
    Products[ProductName] = "Laptop"  
)
```

Q6. SQL Q1. Average salary department-wise [Easy]

```
SELECT department, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department;
```

Q7. SQL Q2. Employee name and manager name using self-join (emp_id, name, manager_id) [Medium]

```
SELECT e.name AS employee,  
       m.name AS manager  
FROM employees e  
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

LEFT JOIN keeps the CEO (manager_id NULL) in the result with a NULL manager.

Q8. SQL Q3. Newest joiner in every department (LEAD/LAG family) [Medium]

```
SELECT * FROM (
    SELECT name, department, join_date,
           ROW_NUMBER() OVER (PARTITION BY department
                              ORDER BY join_date DESC) AS rn
    FROM employees
) t
WHERE rn = 1;
```

ROW_NUMBER partitioned by department, ordered by join_date descending — rn = 1 is the newest joiner per department.

Q9. Python Q1. Create a dictionary, add, modify, and print in alphabetical order of keys [Easy]

```
d = {"banana": 2, "apple": 5}
d["cherry"] = 7           # add
d["apple"] = 10           # modify
for k in sorted(d):       # alphabetical keys
    print(k, d[k])
```

Q10. Python Q2. Unique values in a list and their counts [Easy]

```
from collections import Counter
nums = [1, 2, 2, 3, 3, 3, 4]
counts = Counter(nums)
for value, cnt in counts.items():
    print(value, "appears", cnt, "times")
```

Q11. Python Q3. Find and print duplicate values with their counts [Easy]

```
from collections import Counter
nums = [1, 2, 2, 3, 3, 3, 4]
for value, cnt in Counter(nums).items():
    if cnt > 1:
        print(value, "is duplicated", cnt, "times")
```

Deloitte

Q1. Explain step-by-step how you will create a sales dashboard from scratch [Medium]

- **Requirements:** meet stakeholders — which KPIs, which grain, who will use it.
- **Data:** connect sources (SQL/Excel), profile the data.
- **Clean:** Power Query — types, duplicates, nulls, unpivot targets.
- **Model:** star schema — Sales fact + Date/Product/Customer/Region dims, mark the date table.
- **DAX:** Total Sales, Profit %, YoY, YTD measures.
- **Design:** KPI cards on top, trends middle, detail matrix below; slicers; drill-through; consistent theme.
- **Validate:** tie numbers to source reports.
- **Deploy:** publish, gateway + scheduled refresh, RLS, share via app; collect feedback and iterate.

Q2. Explain how you can optimize a slow Power BI report [Hard]

Layered answer: data model (star schema, drop unused/high-cardinality columns, integer keys, single-direction relationships) → DAX (measures over calculated columns, avoid row-by-row FILTER when a boolean filter works) → source (query folding, incremental refresh, aggregations) → report (fewer visuals, page filters, reduce interactions) → diagnose with Performance Analyzer and fix the top offenders first.

Q3. Explain any 5 chart types and their uses [Easy]

- **Bar/Column:** compare values across categories (sales by region).
- **Line:** trends over time (monthly revenue).
- **Pie/Donut:** composition — parts of a whole, few categories.
- **Scatter:** relationship between two measures, spotting outliers (discount vs profit).
- **Matrix/Heatmap:** values across two dimensions with conditional color (region × month sales intensity).

Q4. SQL Q1. RANK() vs DENSE_RANK() vs ROW_NUMBER() with example [Medium]

For salaries 100, 100, 90: ROW_NUMBER → 1,2,3 · RANK → 1,1,3 (gap) · DENSE_RANK → 1,1,2 (no gap).


```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,
       RANK() OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
FROM employee;
```

Q5. SQL Q2. Find the nth highest salary from Employee [Medium]

```
SELECT salary FROM (
  SELECT salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employee
) t
WHERE rnk = @n; -- e.g. 3 for 3rd highest
```

Q6. SQL Q3. All employees under a manager, including subordinates at any level (hierarchy) [Hard]

```
WITH team AS (
  SELECT EmpID, ManagerID
  FROM employee
  WHERE ManagerID = @manager_id -- direct reports
  UNION ALL
  SELECT e.EmpID, e.ManagerID
  FROM employee e
  JOIN team t ON e.ManagerID = t.EmpID -- their reports, recursively
)
SELECT * FROM team;
```

A **recursive CTE** — the anchor gets direct reports; the recursive part walks down the tree until no more levels.

Q7. SQL Q4. Cumulative salary department-wise for employees who joined in the last 30 days [Hard]

```
SELECT Dept, EmpID, JoinDate, Salary,
       SUM(Salary) OVER (PARTITION BY Dept
                        ORDER BY JoinDate
                        ROWS UNBOUNDED PRECEDING) AS cumulative_salary
FROM employee
WHERE JoinDate >= DATEADD(DAY, -30, GETDATE());
```

Q8. SQL Q5. Top 2 customers by order amount per product category, handling ties [Hard]

```
SELECT * FROM (
  SELECT CustomerID, ProductCategory, OrderAmount,
         DENSE_RANK() OVER (PARTITION BY ProductCategory
                        ORDER BY OrderAmount DESC) AS rnk
  FROM customer
) t
WHERE rnk <= 2;
```

DENSE_RANK keeps tied customers together — if two tie at rank 1, both appear (that's "handling ties appropriately").

Q9. Behavioral Q1. Why do you want to become a data analyst and why this company? [Easy]

Formula: genuine origin story (what pulled you to data) + proof of commitment (projects/upskilling) + 1–2 researched, specific reasons for the company (their domain, clients, culture). Avoid generic lines like "big brand name".

Q10. Behavioral Q2. A difficult task with tight deadlines — how did you handle it? [Medium]

Answer in **STAR**: Situation (context) → Task (what was needed by when) → Action (prioritized, broke work down, communicated early, automated a step) → Result (delivered on time + a number). Pick a real story; interviewers probe details.

TCS

Q1. Write a DAX to calculate the running total monthly [Medium]

```
Running Total Monthly =
CALCULATE(
  SUM(Sales[Amount]),
  FILTER(
    ALLSELECTED('Date'),
    'Date'[Date] <= MAX('Date'[Date])
  )
)
```

```
)  
)
```

Or year-bounded: `TOTALYTD(SUM(Sales[Amount]), 'Date'[Date])`.

Q2. Explain the bookmark in Power BI [Easy]

A bookmark captures the state of a page — filters, slicers, visual visibility, sort. Combined with buttons and the Selection pane, bookmarks build toggle views (chart ↔ table), pop-up filter panels, and guided story navigation. Create via View → Bookmarks → Add.

Q3. Difference between SUM and SUMX? [Medium]

`SUM(column)` aggregates one existing column. `SUMX(table, expression)` is an **iterator** — evaluates the expression row by row then sums, needed when the value must be computed per row first, e.g. `SUMX(Sales, Sales[Qty] * Sales[Price])`. `SUM` is faster; use `SUMX` only when row-level math is required.

Q4. Difference between ALL and ALLSELECTED? [Hard]

`ALL` ignores **every** filter (true grand total); `ALLSELECTED` ignores filters inside the visual but respects user selections (slicers). % of grand total → `ALL`; % of the user's current selection → `ALLSELECTED`.

Q5. SQL Q1. Difference between clustered and non-clustered index? [Medium]

Clustered: defines the physical order of table rows — one per table (usually the primary key); the table *is* the index.

Non-clustered: a separate structure with pointers to rows — many allowed per table; great for frequent `WHERE`/`JOIN` columns. Analogy: clustered = dictionary order, non-clustered = index at the back of a book.

Q6. SQL Q2. Difference between CTE and Views? [Medium]

CTE: temporary named result inside one query — vanishes after execution, supports recursion, improves readability. **View:** a saved query object in the database — reusable across queries/users, can have permissions, can be indexed (materialized). Rule: one-off logic → CTE; reusable logic → view.

Q7. SQL Q3. For 'A,B,B,C,C,C,D' give ROW_NUMBER, DENSE_RANK and RANK [Medium]

```
- Value | ROW_NUMBER | RANK | DENSE_RANK  
- A | 1 | 1 | 1  
- B | 2 | 2 | 2  
- B | 3 | 2 | 2  
- C | 4 | 4 | 3  
- C | 5 | 4 | 3  
- C | 6 | 4 | 3  
- D | 7 | 7 | 4
```

`ROW_NUMBER` never repeats; `RANK` repeats and skips; `DENSE_RANK` repeats without gaps.

Q8. SQL Q4. Query for 2nd highest salary [Easy]

```
SELECT MAX(salary) FROM employee  
WHERE salary < (SELECT MAX(salary) FROM employee);  
-- or  
SELECT DISTINCT salary FROM employee  
ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

EATON

Q1. L1-1. Difference between star and snowflake schema [Easy]

Star: denormalized dimensions directly around the fact table — fewer joins, faster, BI-friendly. Snowflake: dimensions normalized into sub-tables — saves storage, more joins, used for very large dimensions.

Q2. L1-2. How complex queries have you written in SQL? [Medium]

Describe your genuinely hardest query: multi-CTE pipeline, window functions for ranking/running totals, self-join for hierarchy, conditional aggregation with `CASE` — walk through one real example and why it was complex.

Q3. L1-3. Which functions do you use most in SQL? [Easy]

Honest analyst answer: aggregate functions (SUM/COUNT/AVG) with GROUP BY, JOINs daily, window functions (ROW_NUMBER, DENSE_RANK, LAG), CASE WHEN, date functions (DATEADD/DATEDIFF), string functions (CONCAT, SUBSTRING, TRIM) and COALESCE for nulls.

Q4. L1-4. In which domain have you worked? [Easy]

State your project domain (retail, banking, healthcare, manufacturing...) and add one domain-specific metric you handled (e.g. inventory turnover for retail) — that one detail makes it credible.

Q5. L1-5. How to normalize data? [Medium]

Apply normal forms step by step: **1NF** — atomic values, no repeating groups; **2NF** — remove partial dependencies on a composite key; **3NF** — remove transitive dependencies (non-key depending on non-key). Result: each fact stored once, linked via keys.

Q6. L1-6. Difference between normalized and denormalized data [Easy]

Normalized: many small related tables, no redundancy — best for OLTP writes and integrity. Denormalized: merged wider tables with some duplication — fewer joins, faster reads, standard for analytics/warehouses (star schema is deliberate denormalization).

Q7. L1-7. Do you know Power Automate and Power Apps? [Easy]

Ideal answer: "Working knowledge — Power Automate for flows like refresh-completion alerts or emailing reports, Power Apps for simple data-entry apps that write back to a source Power BI reads. I integrate them with Power BI within the Power Platform." Be honest about depth.

Q8. L2-1. What are the types of dimensions? [Hard]

Conformed (shared across facts, e.g. Date), **Slowly Changing** (attributes change over time — SCD types), **Junk** (mixed low-cardinality flags), **Degenerate** (e.g. invoice number stored in the fact), **Role-playing** (one Date dim used as Order/Ship/Delivery date).

Q9. L2-3. Difference between data mart and dataflow [Hard]

Data mart: a subject-focused slice of the warehouse (Sales mart) for one department. In Power BI, **Dataflow** = reusable cloud Power Query ETL (entities stored in the service), while Power BI's **Datamart** feature = dataflow + managed SQL database + dataset in one self-service package.

Q10. L2-4. Have you applied dynamic RLS? Which DAX function? [Hard]

Yes — dynamic RLS uses USERPRINCIPALNAME() (or USERNAME()) in the role rule, matched against a user-access table: [Email] = USERPRINCIPALNAME(). One rule serves every user via the mapping table.

Q11. L2-5. What is SCD and its types? Explain briefly [Hard]

Slowly Changing Dimensions — how dimension attribute changes are stored: **Type 0** never change; **Type 1** overwrite (no history); **Type 2** add a new row with start/end dates + current flag (full history — most used); **Type 3** previous-value column (limited history); higher types combine these.

Virtusa Consulting

Q1. SQL 1. How do you calculate a running total in SQL? [Medium]

```
SELECT order_date, amount,
       SUM(amount) OVER (ORDER BY order_date
                        ROWS UNBOUNDED PRECEDING) AS running_total
FROM orders;
```

Q2. SQL 2. How can you retrieve last year's revenue? Which function? [Medium]

```
SELECT SUM(amount) AS last_year_revenue
FROM orders
WHERE YEAR(order_date) = YEAR(GETDATE()) - 1;
-- or with LAG for a year-over-year table:
SELECT yr, revenue,
       LAG(revenue) OVER (ORDER BY yr) AS prev_year_revenue
```

```
FROM yearly_sales;
```

Q3. SQL 3. How do you perform period comparison in SQL or BI tools? [Medium]

SQL: aggregate per period and use LAG() to bring the previous period onto the same row, then compute the difference/%.
Power BI: time-intelligence DAX — SAMEPERIODLASTYEAR, DATEADD, PARALLELPERIOD against a proper date table.

Q4. SQL 4. Explain the use of LEAD and LAG functions [Medium]

They read another row's value without a self-join: LAG(x) = previous row, LEAD(x) = next row (per ORDER BY, optionally per PARTITION). Classic uses: month-over-month change, days between consecutive orders, comparing a row with the next event.

Q5. SQL 5–6. Filter/sort top 10 records based on a metric field [Easy]

```
SELECT TOP 10 * FROM sales ORDER BY revenue DESC;      -- SQL Server
SELECT * FROM sales ORDER BY revenue DESC LIMIT 10;    -- MySQL/Postgres
```

For top 10 *per group*, use ROW_NUMBER() OVER (PARTITION BY group ORDER BY revenue DESC) and filter ≤ 10.

Q6. SQL 7. Current date sales compared with last year's sales [Medium]

```
SELECT
  SUM(CASE WHEN CAST(order_date AS DATE) = CAST(GETDATE() AS DATE)
        THEN amount END) AS today_sales,
  SUM(CASE WHEN CAST(order_date AS DATE) =
        CAST(DATEADD(YEAR, -1, GETDATE()) AS DATE)
        THEN amount END) AS same_day_last_year
FROM orders;
```

Q7. SQL 8. Is rollup possible using grouping and aggregation? [Hard]

Yes — GROUP BY ROLLUP(region, category) produces subtotals per region and a grand total in one query; CUBE gives all combinations; GROUPING() identifies subtotal rows.

Q8. SQL 9. How do you calculate SUM(X) and sum of differences? [Easy]

```
SELECT SUM(x) AS total_x,
       SUM(x - y) AS total_difference  -- = SUM(x) - SUM(y)
FROM t;
```

Q9. SQL 10. Difference between ALL and ALLEXCEPT (DAX)? [Hard]

ALL removes filters from an entire table/column. ALLEXCEPT(table, col1...) removes all filters **except** the listed columns — e.g. total per region ignoring everything else: CALCULATE(SUM(Sales[Amt]), ALLEXCEPT(Sales, Sales[Region])).

Q10. SQL 11. Explain joins and constraints in SQL [Easy]

Joins: INNER (matches only), LEFT/RIGHT (keep one side), FULL (both), CROSS (cartesian), SELF (table with itself).

Constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT — rules that protect data integrity.

Q11. SQL 12. What is a temporary table? [Easy]

A table that lives only for the session (#temp in SQL Server) or transaction — used to store intermediate results reused across multiple statements, e.g. staging a filtered dataset before several analyses.

Q12. SQL 13. How do you delete duplicate records from a table? [Medium]

```
WITH ranked AS (
  SELECT id, ROW_NUMBER() OVER (
    PARTITION BY email ORDER BY id) AS rn
  FROM customers
)
DELETE FROM customers
WHERE id IN (SELECT id FROM ranked WHERE rn > 1);
```

Q13. SQL 14. Difference between a CTE and a View? [Medium]

CTE — temporary, exists only inside one query, supports recursion. View — a stored, reusable query object with permissions; indexed views can even persist results. One-off readability → CTE; shared reusable logic → View.

Q14. SQL 15. A stored procedure runs for over an hour — how do you reduce execution time? [Hard]

- Get the **execution plan**; find scans, spills and heavy operators.
- Add/repair **indexes** on join/filter columns; update statistics.
- Replace cursors/row-by-row logic with **set-based** operations.
- Break giant queries into indexed temp-table steps; filter early; remove SELECT *.
- Check **parameter sniffing** (OPTION(RECOMPILE) / local variables), blocking and tempdb pressure.

Q15. PBI 1. What is Mixed (Composite) Mode in Power BI? [Medium]

A model that combines **Import** and **DirectQuery** sources: small dimensions imported for speed, huge fact tables on DirectQuery for freshness, with dual-storage tables bridging both. Best-of-both approach for big data.

Q16. PBI 2. Two tables — one updates dynamically, the other stays static? [Medium]

Use a composite model: the dynamic table on **DirectQuery** (always current) and the static one on **Import**. Alternative: both Import but exclude the static table from refresh ("Include in report refresh" off in Power Query).

Q17. PBI 3. In MS Fabric, DirectQuery vs Direct Lake? [Hard]

DirectQuery sends live SQL to the source per interaction — always fresh, slower. **Direct Lake** (Fabric) reads Delta/Parquet files in OneLake directly into the VertiPaq engine — near-import speed *without* copying/refreshing data. Direct Lake ≈ import performance + DirectQuery freshness.

Q18. PBI 4. Import vs DirectQuery — which is better, in what scenarios? [Medium]

Import is better for most reports: fastest visuals, full DAX/Power Query. **DirectQuery** when data is too large to import, must be real-time, or must stay in the source for compliance. Composite when you need both.

Q19. PBI 5. Explain the VertiPaq engine [Hard]

Power BI's in-memory **columnar storage engine** behind Import mode: stores data column-wise, heavily compressed (dictionary + run-length encoding), scans only needed columns — that's why imported models with fewer, low-cardinality columns fly.

Q20. PBI 6. What is cross-filter direction? [Medium]

A relationship setting: **Single** — filters flow one way (dimension → fact; the recommended default) or **Both** — filters flow both ways (needed for some many-to-many cases, but risks ambiguity and slowness — use sparingly).

Q21. Model 1. Customer linked to sales, each customer belongs to a region — model it? [Medium]

Star schema: Region (1) → Customer (many) as a snowflaked arm, or better flatten region attributes into the Customer dimension → Customer (1) → Sales (many). Filters then flow Region → Customer → Sales naturally.

Q22. Model 2. Count of distinct products sold? [Easy]

```
Distinct Products Sold = DISTINCTCOUNT(Sales[ProductID])
```

In SQL: COUNT(DISTINCT product_id). In DAX, DISTINCTCOUNT on the fact table's product key counts only products that actually appear in sales.

Q23. Model 3. Sales linked to Products but no direct relationship — how to handle? [Hard]

Options: create the relationship on a shared key if one exists; use a **bridge table** for many-to-many; or filter virtually in DAX with TREATAS(VALUES(Products[ID]), Sales[ProductID]) when a physical relationship isn't possible.